

PROGRAMMAZIONE II

CLASSI ANNIDATE E ANONIME

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**

Dipartimento
di **INFORMATICA**

A.A. 2025/2026

CLASSI ANNIDATE



CLASSE ANNIDATA

Una classe (statica) può essere definita all'interno di un'altra classe

// *chiamata annidata*

```
package pkg;
class Outer {
    static class Nested {
        ...
    }
}
```

1
2
3
4
5
6

Simile alle classi ordinarie

- ▶ Stesse regole di visibilità
- ▶ Il nome completo include la classe esterna: `pkg.Outer.Nested`

CLASSE ANNIDATA (CONT.)

Una classe (statica) può essere definita all'interno di un'altra classe

// chiamata annidata

Sono membri statici della classe racchiudente

- ▶ Hanno accesso agli altri membri `static` della classe racchiudente *// anche private*
- ▶ Possono essere usate indipendentemente dalla classe racchiudente

```
// crea un oggetto per la classe annidata
Outer.Nested nestedObject = new Outer.Nested();
```

1

2

CLASSE INTERNA

Una classe (non statica) può essere definita all'interno di un'altra classe

// *chiamata interna*

```
package pkg;  
class Outer {  
    class Inner {  
        ...  
    }  
}
```

1
2
3
4
5
6

Simile alle classi ordinarie

- ▶ Stesse regole di visibilità
- ▶ Il nome completo include la classe esterna: `pkg.Outer.Inner`

CLASSE INTERNA (CONT.)

Una classe (non statica) può essere definita all'interno di un'altra classe

// chiamata interna

Un'istanza di una classe interna non può esistere all'esterno di un'istanza della classe esterna

- ▶ Hanno accesso agli altri membri della classe racchiudente *// anche private*
- ▶ Non possono essere usate indipendentemente dalla classe racchiudente

```
// crea un oggetto per la classe interna
Outer outerObject = new Outer();
Outer.Inner innerObject = Outer.new Inner();
```

1

2

3

CLASSI ANNIDATE VS CLASSI INTERNE

```
class Outer {                                     1
    private static int I = 5;                     2
    private int i;                                3
                                                    4
    static class Nested { // non ha accesso al membro di istanza i 5
        static void print() { System.out.println(I); } 6
    }                                              7
    class Inner { // ha accesso al membro di istanza i 8
        void print() { System.out.println(i); } 9
    }                                             10
                                                    11
    public void outerMethod() {                   12
        Nested.print();                           13
        Inner innerObject = new Inner();          14
        innerObject.print();                       15
    }                                              16
}                                                  17
```

CLASSE LOCALE

Una classe non statica può essere definita all'interno del metodo di un'altra classe // *chiamata locale*

- L'istanziamento e l'accesso alle variabili sono limitati dal metodo racchiudente

```
public void outerMethod() {
    int j = 1;

    class Local { // nessun modificatore di accesso
        int addOne() { return j + 1; }
    }

    Local localObject = new Local();
    System.out.println(localObject.addOne()); // l'output è '2'
}
```

1
2
3
4
5
6
7
8
9
10

CLASSE LOCALE (CONT.)

Una classe non statica può essere definita all'interno del metodo di un'altra classe // *chiamata locale*

- Può accedere solo a variabili **final** // *una sorta di closure*

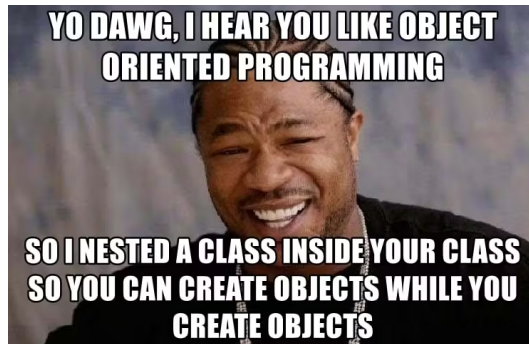
```
public void outerMethod() {
    final int j = 1;

    class Local { // nessun modificatore di accesso
        int addOne() { return j + 1; } // j++ non consentito
    }

    Local localObject = new Local();
    System.out.println(localObject.addOne()); // l'output è '2'
}
```

1
2
3
4
5
6
7
8
9
10

OGGETTI DENTRO OGGETTI



PERCHÉ ANNIDARE LE CLASSI?

Migliore organizzazione del codice

- ▶ A volte è preferibile inserire una classe nel namespace di un'altra classe

Politica di visibilità più semplice

- ▶ Le classi annidate hanno un accesso speciale alle variabili della classe racchiudente

Base di codice meno complessa

- ▶ Nessuna necessità di creare un file apposito per classi con ambito limitato

PERCHÉ ANNIDARE LE CLASSI? (CONT.)

```
class Classroom implements Iterable {
    private Student[] students;

    Classroom(Student[] students)
    { this.students = students; }
    @Override
    public Iterator iterator()
    { return new CRIterator(); }
}
```

```
class CRIterator implements Iterator {
    private Student[] arr; private int next = 0;

    CRIterator(Student[] arr) { this.arr = arr; }

    @Override
    public boolean hasNext()
    { return next < arr.length; }
    @Override
    public Object next()
    { return new Student(arr[next++]); }
}
```

```
class Classroom implements Iterable {
    private Student[] students;

    Classroom(Student[] students) {
        this.students = students;
    }
    @Override
    public Iterator iterator() {
        return new CRIterator(students);
    }
}
```

```
class CRIterator implements Iterator {
    private int next = 0;

    @Override
    public boolean hasNext() {
        return next < students.length;
    }
    @Override
    public Object next() {
        return new Student(students[next++]);
    }
}
```

CLASSI ANONIME



CLASSE ANONIMA

Una classe locale priva di nome

- ▶ Tipicamente, le classi locali vengono istanziate e usate una sola volta
- ▶ Possibile solo mediante ereditarietà

Estendere una classe

```
new SomeParentClass( argomenti del costruttore ) { dichiarazioni dei metodi }
```

```
new Person("Sam") {  
    @Override  
    public String toString() { return "It's Sam!"; }  
}
```

1
2
3
4

CLASSE ANONIMA (CONT.)

Una classe locale priva di nome

- ▶ Tipicamente, le classi locali vengono istanziate e usate una sola volta
- ▶ Possibile solo mediante ereditarietà

Implementare un'interfaccia

```
new SomeInterface() { dichiarazioni dei metodi }
```

```
new Runnable() {  
    @Override  
    public void run() { ... }  
}
```

1
2
3
4

USO DELLE CLASSI ANONIME

Le classi anonime sono espressioni

- Devono fare parte di un'istruzione

Lato sinistro di un assegnamento

```
Person sam = new Person("Sam") {  
    @Override  
    public String toString() { return "It's ␣ Sam!"; }  
};  
System.out.println(sam.toString());
```

Parametro effettivo di un metodo

```
List<Runnable> tasks = new ArrayList<>();  
tasks.add(new Runnable() {  
    @Override  
    public void run() { ... }  
});
```



Non possiamo specificare un costruttore per le classi anonime

LAMBDA

Classi anonime semplificate, che implementano un'interfaccia a [singolo metodo](#)

```
public interface SingleInterface { int singleMethod(int param); }
```

```
SingleInterface singleInc = new SingleInterface() { // classe anonima      1
    @Override                                          2
    public int singleMethod(int param) { return param + 1; }                3
};                                                    4
SingleInterface singleDec = param -> param - 1; // con una lambda          5
                                                    6
System.out.println(singleInc.singleMethod(3)); // l'output è '4'           7
System.out.println(singleDec.singleMethod(3)); // l'output è '2'           8
```

SINTASSI DELLE LAMBDA

parametri $\rightarrow$ *corpo*

Dove i *parametri* possono essere

- ▶ Zero: `()`
- ▶ Uno: `x`
- ▶ Due o più: `(x, y)`

// i tipi vengono dedotti automaticamente

E il *corpo* può essere

- ▶ Un'espressione: `x + y`
- ▶ Un blocco di codice: `{ return x + y; }`

PERCHÉ LE LAMBDA?

Rendono il codice più conciso e leggibile

- ▶ Focalizzazione sulla funzionalità piuttosto che sull'implementazione
- ▶ Aggiungono un tocco di programmazione funzionale

Hanno comunque delle limitazioni rispetto alle classi anonime

- ▶ Non è possibile definire metodi multipli
- ▶ Intrinsecamente prive di stato, nessuna variabile locale può essere dichiarata

VARARGS

È possibile passare un numero **variabile** di argomenti a un metodo

methodName(*tipo*... args)

```
static int min(int... values) { // values viene convertito in un array      1
    int res = Integer.MAX_VALUE; // con la dimensione corretta              2
    for (int v : values)           3
        if (v < res) res = v;      4
    return res;                    5
}                                  6

public static void main(String[] args) {                                   7
    System.out.println(min(1, 0, 5)); // l'output è '0'                      8
    System.out.println(min(3, 7, 2, 6, 9)); // l'output è '2'                9
}                                  10
                                   11
```

VARARGS (CONT.)

Riducono il codice boilerplate, utilizzabili nelle lambda e nelle classi anonime

```
public interface SingleInterface { int singleMethod(int... params); }
```

```
SingleInterface singleInc = new SingleInterface() { // classe anonima      1
    @Override                                           2
    public int singleMethod(int... params) { return params[0] + 1; }      3
};                                                       4
SingleInterface singleDec = params -> params[0] - 1; // con una lambda    5
                                                       6
System.out.println(singleInc.singleMethod(3, 4, 5)); // l'output è '4'    7
System.out.println(singleDec.singleMethod(3, 2));     // l'output è '2'    8
```

DOMANDE E RISPOSTE

